Continuous Batching Explained

NAN LIN, Shanghai Jiao Tong University, China

Large Language Model (LLM) serving is bottlenecked not only by raw compute, but by *shape rigidity* (rectangular tensors), *memory pressure* (KV cache growth), and *work inefficiency* (padding and wasted decoding after early termination). This article derives **continuous batching** from first principles: starting with causal self-attention, we motivate KV caching, then explain why chunked prefill is necessary under memory constraints, and finally show how ragged batching plus dynamic scheduling eliminates padding waste and keeps GPU utilization high. We close by discussing paged attention as the next step: turning KV cache management into an allocation problem rather than a tensor-shape problem.

1 Introduction

Serving large language models (LLMs) is fundamentally a *scheduling* problem rather than a pure kernel-optimization problem. Training assumes large, homogeneous, and synchronized batches, whereas inference operates under an adversarial workload: requests arrive asynchronously, prompt lengths are highly variable, and generation lengths are unknown until an `<eos>` token is produced. The resulting workload is irregular, stateful (due to autoregressive context), and latency-sensitive.

This mismatch creates three systematic sources of waste in conventional *rectangular* batching. **(1) Padding waste:** variable-length prompts are padded to a common maximum length, inducing redundant attention computation. **(2) Early-termination waste:** completed sequences keep occupying batch slots, forcing useless tokens to be computed until the longest request finishes. **(3) Recomputation waste:** during decoding, naïvely recomputing attention projections for all past tokens induces quadratic redundancy across time. These inefficiencies reduce effective utilization and often shift serving from compute-bound behavior to memory-bound underutilization on modern GPUs.

Key idea. Continuous batching is best understood as a consequence of the *structural asymmetry* of causal attention: *future tokens cannot influence past tokens.* This one-way dependency enables (i) reusing past states (KV cache), (ii) processing long prompts incrementally (chunked prefill), and (iii) packing tokens from heterogeneous requests without padding (ragged batching), while (iv) dynamically admitting and evicting requests to avoid idle capacity. This viewpoint follows the first-principles derivation in the Hugging Face exposition and related serving systems (e.g., vLLM). [**? ?**]

Scope and roadmap. We decompose continuous batching into four orthogonal mechanisms: KV cache, chunked prefill, ragged batching, and dynamic scheduling. We explain how each mechanism arises naturally from causal structure, why their composition preserves semantics, and how the overall system systematically eliminates structural waste, leaving KV-cache memory management as the next dominant bottleneck.

2 Background

2.1 Causal self-attention and one-way dependency

Given token embeddings $X \in \mathbb{R}^{B\times S\times H}$, a single attention head computes

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

Author's Contact Information: Nan LIN, Shanghai Jiao Tong University, Shanghai, China, lns_brandon@sjtu.edu.cn.

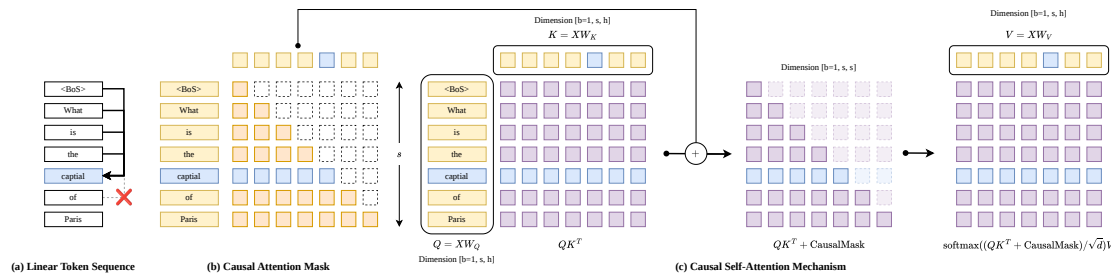

Fig. 1. **Causal self-attention.** The lower-triangular mask enforces autoregressive dependency: future tokens cannot influence past tokens.

and

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + M\right) V, \tag{1}$$

where M is the attention mask. Under **causal masking**, M enforces a lower-triangular constraint: token i may only attend to tokens $j \leq i$. Figure 1 illustrates this autoregressive dependency.

A direct consequence is the **immutability of past**: once keys/values for earlier tokens are computed, later tokens do not change them. This is the fundamental enabler of incremental generation and state reuse. [**?**]

2.2 Prefill vs. decode and KV-cache reuse

Autoregressive generation typically has two phases: **prefill** (processing the entire prompt to initialize internal states) and **decode** (generating new tokens one-by-one). Naïvely, decode would recompute K, V for all previous tokens at every step. However, causal attention implies past K, V are reusable: we can cache them and compute K, V only for newly generated tokens, appending them to the cache. This reduces the incremental work from "recompute everything" to "compute one token and read the cache", trading compute for memory bandwidth. [**?**]

2.3 Why rectangular batching is misaligned with serving

Rectangular batching forms tensors of shape $X \in \mathbb{R}^{B \times S_{\max} \times H}$ by padding all sequences to $S_{\max}$. This representation assumes synchronized progress across requests, which contradicts inference reality: requests have heterogeneous prompt lengths, heterogeneous decode lengths, and terminate at different times. Consequently, padding and idle slots become *structural* waste rather than transient inefficiency. Figure 2 highlights these wasted regions.

3 Continuous Batching

Continuous batching can be viewed as progressively relaxing the constraints that create waste, while preserving the only constraint that matters for correctness: **within-request token order**.

3.1 KV cache: eliminating recomputation

During decoding at step t, only the new token's projections need to be computed, while past (K, V) are read from cache. Figure 3 visualizes that only the "new" token contributes fresh K, V, and all previous states are reused.

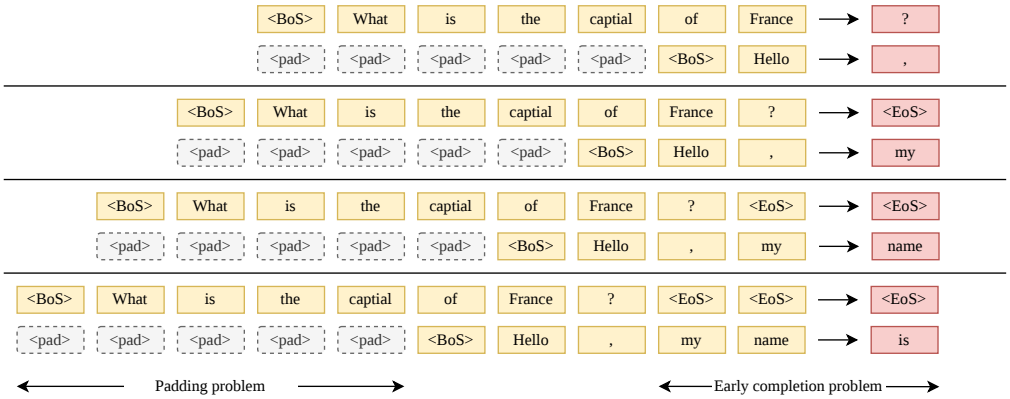

Fig. 2. **Naive batching inefficiencies.** Padding and early termination introduce idle regions that reduce utilization under heterogeneous request lengths.

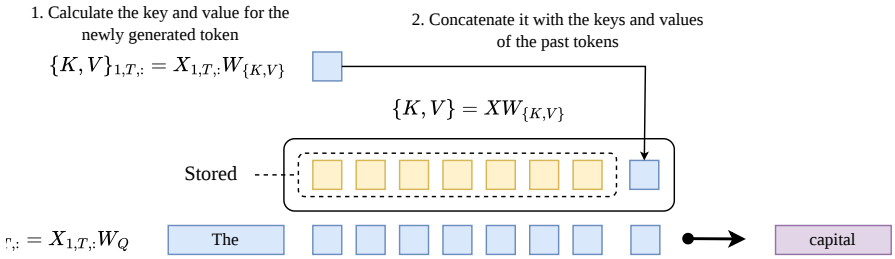

Fig. 3. **KV cache.** Past keys and values are reused across decoding steps; only the projections of newly appended tokens are computed.

3.2 Chunked prefill: memory-bounded prompt processing

Long prompts can exceed the activation/KV memory budget if processed in a single prefill pass. **Chunked prefill** partitions the prompt into segments processed across multiple forward passes. Each segment attends to the cached prefix (via stored K, V) and appends new entries, yielding identical semantics to full prefill because causal attention ensures the prefix does not depend on future tokens. [**?**] Figure 4 shows this incremental accumulation.

3.3 Ragged batching: eliminating padding by construction

Instead of padding sequences to a common length, **ragged batching** concatenates tokens from multiple requests into a single packed dimension, and uses a **block-diagonal** attention mask so tokens from different requests cannot attend to each other. This removes padding waste entirely: every token in the batch corresponds to real work. Figure 5 illustrates the packing and isolation.

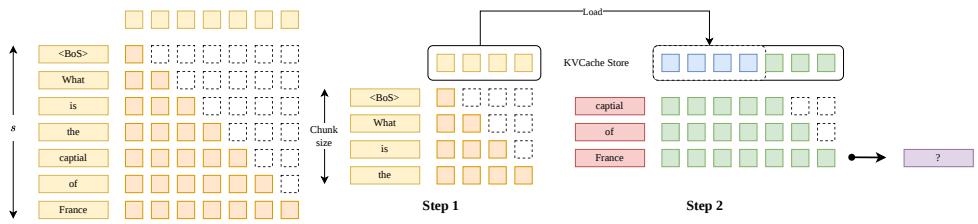

Fig. 4. **Chunked prefill.** Prompt tokens are processed incrementally; cached KV states accumulate across chunks without changing autoregressive semantics.

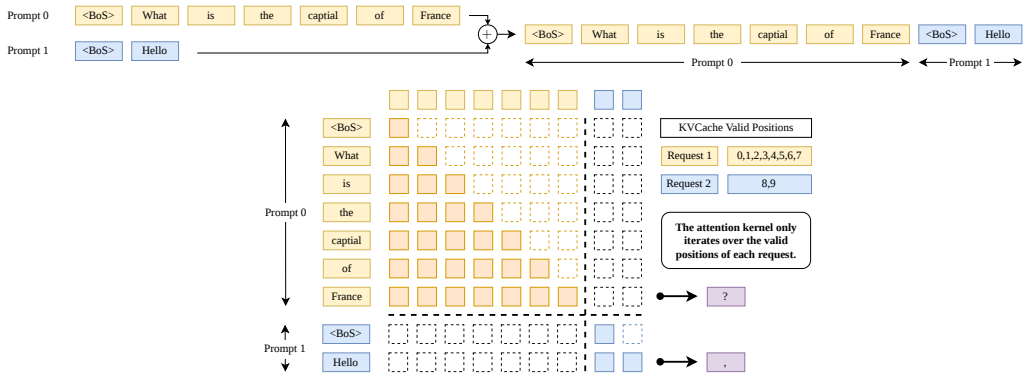

Fig. 5. **Ragged batching.** Tokens from multiple requests are concatenated; a block-diagonal mask enforces isolation, preventing cross-request attention.

3.4 Dynamic scheduling: eliminating idle capacity

Even with ragged batching, a *static* set of requests wastes capacity when some requests terminate earlier. **Dynamic scheduling** treats the batch as a continuously evolving token buffer with a fixed per-step token budget m (bounded by memory/compute). At each scheduling step:

(1) Each active decode request contributes exactly one token (decode is inherently token-serial per request).
(2) Remaining budget is filled with prefill tokens from waiting/new requests (using chunked prefill to fit).
(3) Requests that emit `<eos>` are removed immediately.
(4) Newly arrived (or queued) requests are admitted immediately.

This keeps the device saturated while preserving per-request autoregressive order.

Figure 6 presents a concrete timeline (token budget: $m = 5$) for three requests: R1=`"hello, my"`, R2=`"how is the weather today?"`, and R3=`"Tell me a joke."`. In Step 1, tokens from R1 and the prefix of R2 are packed; R1 can complete prefill and start generating (`name`). In Step 2, R1 decodes one token while R2 continues chunked prefill; R2 reaches `<eos>` and exits. In Step 3, R1 continues decoding, and R3 is admitted with partial (chunked) prefill due to the fixed budget.

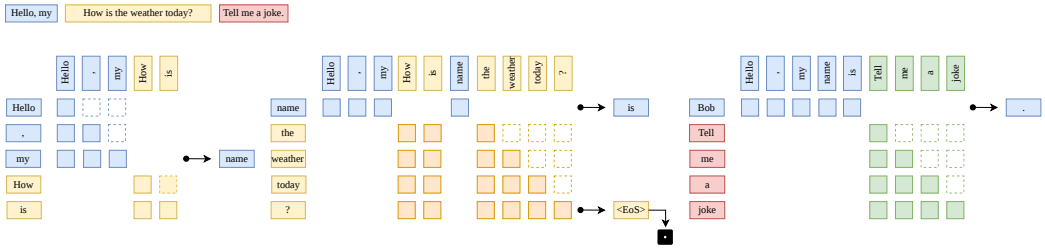

Fig. 6. **Continuous batching under a fixed token budget (m=5).** Decode tokens (one per active request) are interleaved with chunked prefill tokens from queued requests. Requests enter and leave dynamically (e.g., R2 exits upon <eos>) without violating within-request autoregressive order, because cross-request dependencies are prohibited by masking.

3.5 Synthesis

Continuous batching composes four mechanisms:

- **State reuse** (KV cache) removes recomputation across decode steps.
- **Memory-bounded prefill** (chunked prefill) enables incremental prompt processing.
- **Spatial packing** (ragged batching) eliminates padding by construction.
- **Temporal packing** (dynamic scheduling) removes idle slots by admitting/evicting requests immediately.

Together, they preserve model outputs while systematically eliminating structural waste. Once these redundancies are removed, serving commonly transitions toward **memory-bound** behavior, motivating subsequent work on KV-cache memory management (e.g., paged/virtualized KV allocation). [**? ?**]

4 Conclusion

Continuous batching is not an ad-hoc engineering trick: it is a direct consequence of causal attention's one-way dependency. Its correctness follows from (i) the immutability of cached states, and (ii) mask-enforced isolation between requests. Its performance gains follow from removing three classes of waste—padding, early-termination idle slots, and recomputation—at their structural roots. With compute waste minimized, the optimization frontier shifts to KV-cache memory layout and allocation, where paging-inspired designs such as PagedAttention provide a complementary system layer. [**?**]